WEBTECHNOLOGIEN

B5 – FORMULARE

PROF. DR. MARKUS HECKNER

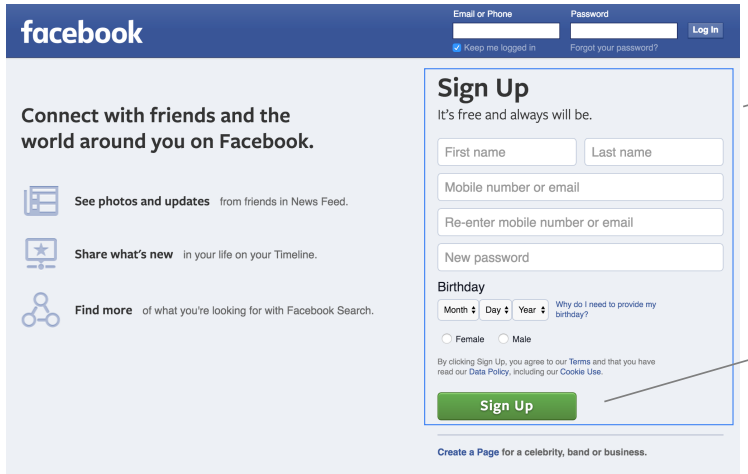
THE STORY SO FAR

- HTML und CSS für das Frontend der Website
- Web-App als dynamischer Webserver, der die aufgerufenen Seiten an den Client zurückgibt, der auf Anfragen reagieren kann und diese nach dem Model View Controller-Prinzip verarbeitet
- PostgreSQL als Datenbank für die Models
- Anfragen an die Web-App nur über URLs, z.B. `/playlist/1` zur Abfrage der Playlist mit der ID 1

Jetzt: Wie können Nutzer Eingaben an die Web-App schicken?

HTML FORMULARE ERMÖGLICHEN DIE ERFASSUNG UND WEITERGABE VON NUTZEREINGABEN

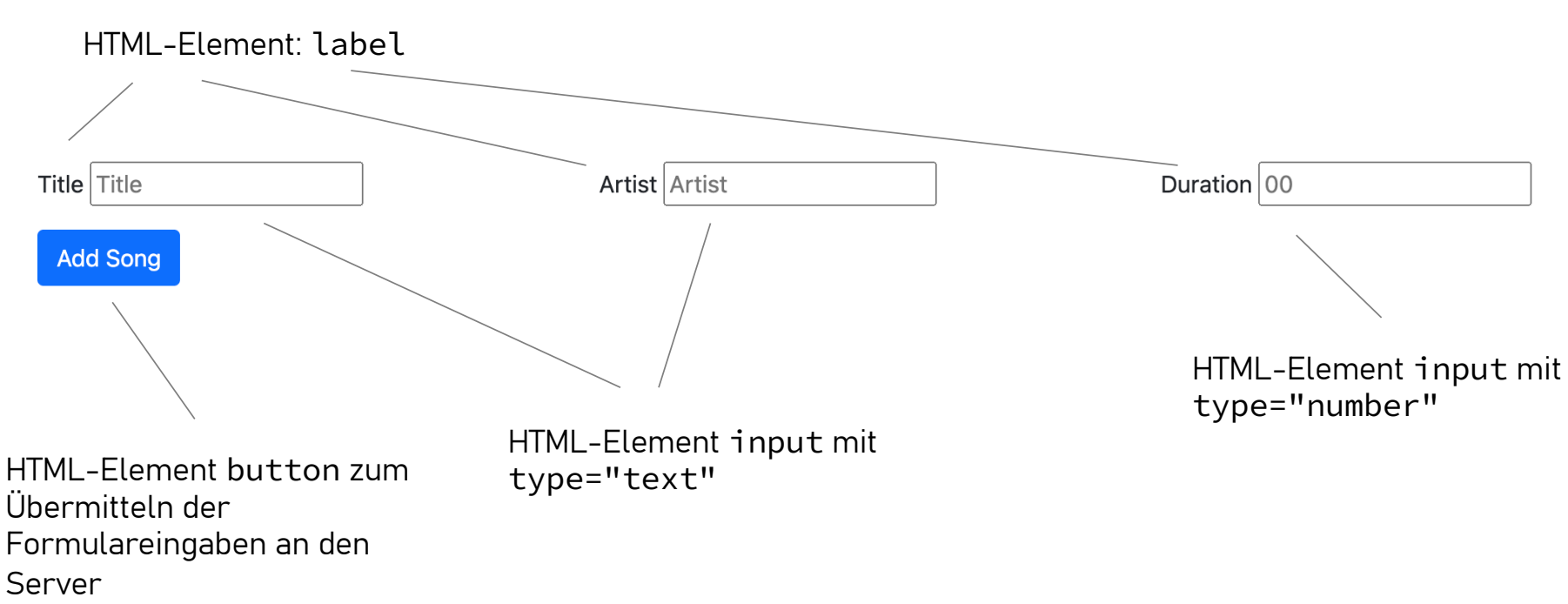
- Interaktionsmöglichkeit für Nutzer mit Webseiten
- Daten werden an Webserver zur weiteren Verarbeitung übermittelt (z.B. als Werte für Abfragen an Datenbanken, Überprüfung von Login-Daten, Bestellen von Artikeln bei Amazon, usw.)
- Formulare bestehen aus HTML Elementen zur Eingabe (Text, Radiobuttons, Checkboxen, ...) und zum Absenden des Formulars (Button)



Formular mit Feldern

Übermitteln der Formular-eingaben an den Server

FORMULAR IM BROWSER ZUM HINZUFÜGEN EINES NEUEN SONGS



FORMULARE ÜBERMITTELN DIE DATEN PER HTTP-POST AN DEN SERVER

- GET – Daten vom Server holen
- POST – Erstellen einer Ressource (Daten auf dem Server ablegen) } Formulare (heute)
- PATCH – Daten auf dem Server aktualisieren
- DELETE – Löschen von Daten auf dem Server

} Nicht in diesem Kurs

- ...

Quelle: https://developer.mozilla.org/de/docs/Web/HTTP/Methods

HANDLEBARS CODE DES FORMULARS (OHNE BOOTSTRAP)

Was passiert, beim Absenden des Formulars?
Hier Aufrufen einer parametrisierten Route (`{{playlist.id}}` wird serverseitig durch die ID der Playlist ersetzt), um einen neuen Song hinzuzufügen

http-Methode POST (wird im Router wichtig)

```
<form action="/playlist/{{playlist.id}}/addsong" method="POST">
    <label>Title</label> <input name="title" type="text" placeholder="Title" />
    <label>Artist</label> <input name="artist" type="text" placeholder="Artist" />
    <label>Duration</label> <input name="duration" type="number" placeholder="00" />
    <button type="submit">Add Song</button>
</form>
```

Eingabe-felder

Schickt das Formular ab, d.h. die in `action` definierte URL wird mit der in `method` definierten http-Methode aufgerufen

`name` ermöglicht es dem Server die Eingaben aus dem Formular auszulesen

Welche Art von Daten wird eingegeben?

HTTP REQUEST HEADER – KURZ UND KNAPP

Wie werden die Daten übermittelt? Hier: POST als Teil des headers...

Request Headers

```
:authority: playlist-2.mheckner.repl.co
:method: POST
:path: /playlist/2/addsong
:scheme: https
accept: text/html,application/xhtml+xml,application/
*;q=0.8,application/signed-exchange;v=b3;q=0.9
accept-encoding: gzip, deflate, br
accept-language: en-US,en;q=0.9,de;q=0.8,la;q=0.7
```

Welchen Pfad wollen wir?

Form Data view parsed

```
title=Song+2&artist=Blur&duration=122
```

Body (Inhalt) des Requests (MIME-Type: `application/x-www-form-urlencoded`): Hier stehen die Daten aus dem Formular als Key-Value Paare (+ = Leerzeichen, & trennt Key-Value Paare)...
Key entspricht dem name-Attribut des Formulars!

ROUTER NIMMT DIE ABFRAGE ENTGEGEN

routes.js:

Nur für POST-requests

```
router.post('/playlist/:id/addsong', playlist.addSong);
```

Route - Parameter `:id` für die Playlist zu der der Song hinzugefügt werden soll.

Wird durch Handlebars ersetzt, bevor das Formular an den Client geschickt wird (vgl. http-Request header)

Aufrufen der Funktion addSong des playlist-Controllers

AUSLESEN DER FORMULARDATEN IM PLAYLIST-CONTROLLER

controllers/playlist.js:

```
async addSong(request, response) {
      const playlistId = request.params.id;
      const newSong = {
        title: request.body.title,
        artist: request.body.artist,
        duration: Number(request.body.duration)
      };
      logger.debug("New Song", newSong);
      await songStore.addSong(playlistId, newSong);
      response.redirect("/playlist/" + playlistId);
},
```

body ist der Inhalt des Requests und enthält die geparsten name-Attribute als Keys – Inhalte sind die Eingaben in den Formularfeldern (man muss den String aus Key-Value-Paaren nicht selbst zerlegen, das macht der body-parser)

SERVER FÜR DAS AUSLESEN DER FORMULARDATEN KONFIGURIEREN

app.js konfigurieren für Auslesen von POST-Daten – Sonst Fehler und Absturz:

```
app.use(express.json());
app.use(express.urlencoded( options: { extended: true }));
```

FORMULARE LASSEN SICH MIT BOOTSTRAP VISUELL ANPASSEN

Email address

We'll never share your email with anyone else.

Password

Check me out

Submit

Vgl. Offizielle Dokumentation von Bootstrap 5: https://getbootstrap.com/docs/5.3/forms/overview/

FAZIT

- Formulare erlauben es Nutzern mit der Web-App zu interagieren (z.B. registrieren, einloggen, ausloggen, einen neuen Song hinzufügen, Songs löschen, etc.)
- Formulare werden in HTML mit dem Tag `<form>` gekennzeichnet
- Jedes Formular enthält ein oder mehrere `<input>`-Elemente, deren Attribut `name` festlegt unter welchem Key die Inhalte der `<input>`-Elemente auf dem Server ausgelesen werden können
- Formulare können mit Bootstrap-Klassen visuell angepasst werden